

Program Header

An executable or shared object file's program header table is an array of structures, each describing a segment or other information the system needs to prepare the program for execution. An object file *segment* contains one or more *sections*, as ["Segment Contents"](#) describes below. Program headers are meaningful only for executable and shared object files. A file specifies its own program header size with the ELF header's `e_phentsize` and `e_phnum` members. See ["ELF Header"](#) in Chapter 4 for more information.

Figure 5-1: Program Header

```
typedef struct {
    Elf32_Word    p_type;
    Elf32_Off     p_offset;
    Elf32_Addr    p_vaddr;
    Elf32_Addr    p_paddr;
    Elf32_Word    p_filesz;
    Elf32_Word    p_memsz;
    Elf32_Word    p_flags;
    Elf32_Word    p_align;
} Elf32_Phdr;

typedef struct {
    Elf64_Word    p_type;
    Elf64_Word    p_flags;
    Elf64_Off     p_offset;
    Elf64_Addr    p_vaddr;
    Elf64_Addr    p_paddr;
    Elf64_Xword   p_filesz;
    Elf64_Xword   p_memsz;
    Elf64_Xword   p_align;
} Elf64_Phdr;
```

p_type

This member tells what kind of segment this array element describes or how to interpret the array element's information. Type values and their meanings appear [below](#).

p_offset

This member gives the offset from the beginning of the file at which the first byte of the segment resides.

p_vaddr

This member gives the virtual address at which the first byte of the segment resides in memory.

p_paddr

On systems for which physical addressing is relevant, this member is reserved for the segment's physical address. Because System V ignores physical addressing for application programs, this member has unspecified contents for executable files and shared objects.

p_filesz

This member gives the number of bytes in the file image of the segment; it may be zero.

p_memsz

This member gives the number of bytes in the memory image of the segment; it may be zero.

p_flags

This member gives flags relevant to the segment. Defined flag values appear [below](#).

p_align

As ``Program Loading'' describes in this chapter of the processor supplement, loadable process segments must have congruent values for `p_vaddr` and `p_offset`, modulo the page size. This member gives the value to which the segments are aligned in memory and in the file. Values 0 and 1 mean no alignment is required. Otherwise, `p_align` should be a positive, integral power of 2, and `p_vaddr` should equal `p_offset`, modulo `p_align`.

Some entries describe process segments; others give supplementary information and do not contribute to the process image. Segment entries may appear in any order, except as explicitly noted [below](#). Defined type values follow; other values are reserved for future use.

Figure 5-2: Segment Types, `p_type`

Name	Value
PT_NULL	0
PT_LOAD	1
PT_DYNAMIC	2
PT_INTERP	3
PT_NOTE	4
PT_SHLIB	5
PT_PHDR	6
PT_TLS	7
PT_LOOS	0x60000000
PT_HIOS	0x6fffffff
PT_LOPROC	0x70000000
PT_HIPROC	0x7fffffff

PT_NULL

The array element is unused; other members' values are undefined. This type lets the program header table have ignored entries.

PT_LOAD

The array element specifies a loadable segment, described by `p_filesz` and `p_memsz`. The bytes from the file are mapped to the beginning of the memory segment. If the segment's memory size (`p_memsz`) is larger than the file size (`p_filesz`), the ``extra'' bytes are defined to hold the value 0 and to follow the segment's initialized area. The file size may not be larger than the memory size. Loadable segment entries in the program header table appear in ascending order, sorted on the `p_vaddr` member.

PT_DYNAMIC

The array element specifies dynamic linking information. See [``Dynamic Section''](#) below for more information.

PT_INTERP

The array element specifies the location and size of a null-terminated path name to invoke as an interpreter. This segment type is meaningful only for executable files (though it may occur for shared objects); it may not occur more than once in a file. If it is present, it must precede any loadable segment entry. See [``Program Interpreter''](#) below for more information.

PT_NOTE

The array element specifies the location and size of auxiliary information. See [``Note Section''](#) below for more information.

PT_SHLIB

This segment type is reserved but has unspecified semantics. Programs that contain an array element of this type do not conform to the ABI.

PT_PHDR

The array element, if present, specifies the location and size of the program header table itself, both in the file and in the memory image of the program. This segment type may not occur more than once in a file. Moreover, it may occur only if the program header table is part of the memory image of the program. If it is present, it must precede any loadable segment entry. See ["Program Interpreter"](#) below for more information.

PT_TLS

The array element specifies the *Thread-Local Storage* template. Implementations need not support this program table entry. See ["Thread-Local Storage"](#) below for more information.

PT_L00S through PT_HI0S

Values in this inclusive range are reserved for operating system-specific semantics.

PT_LOPROC through PT_HIPROC

Values in this inclusive range are reserved for processor-specific semantics. If meanings are specified, the processor supplement explains them.



Unless specifically required elsewhere, all program header segment types are optional. A file's program header table may contain only those elements relevant to its contents.

Base Address

As "Program Loading" in this chapter of the processor supplement describes, the virtual addresses in the program headers might not represent the actual virtual addresses of the program's memory image. Executable files typically contain absolute code. To let the process execute correctly, the segments must reside at the virtual addresses used to build the executable file. On the other hand, shared object segments typically contain position-independent code. This lets a segment's virtual address change from one process to another, without invalidating execution behavior. On some platforms, while the system chooses virtual addresses for individual processes, it maintains the *relative* position of one segment to another within any one shared object. Because position-independent code on those platforms uses relative addressing between segments, the difference between virtual addresses in memory must match the difference between virtual addresses in the file. The differences between the virtual address of any segment in memory and the corresponding virtual address in the file is thus a single constant value for any one executable or shared object in a given process. This difference is the *base address*. One use of the base address is to relocate the memory image of the file during dynamic linking.

An executable or shared object file's base address (on platforms that support the concept) is calculated during execution from three values: the virtual memory load address, the maximum page size, and the lowest virtual address of a program's loadable segment. To compute the base address, one determines the memory address associated with the lowest `p_vaddr` value for a `PT_LOAD` segment. This address is truncated to the nearest multiple of the maximum page size. The corresponding `p_vaddr` value itself is also truncated to the nearest multiple of the maximum page size. The base address is the difference between the truncated memory address and the truncated `p_vaddr` value.

See this chapter in the processor supplement for more information and examples. "Operating System Interface" of Chapter 3 in the processor supplement contains more information about the virtual address space and page size.

Segment Permissions

A program to be loaded by the system must have at least one loadable segment (although this is not required by the file format). When the system creates loadable segments' memory images, it gives access permissions as

specified in the `p_flags` member.

Figure 5-3: Segment Flag Bits, `p_flags`

Name	Value	Meaning
PF_X	0x1	Execute
PF_W	0x2	Write
PF_R	0x4	Read
PF_MASKOS	0x0ff00000	Unspecified
PF_MASKPROC	0xf0000000	Unspecified

All bits included in the `PF_MASKOS` mask are reserved for operating system-specific semantics.

All bits included in the `PF_MASKPROC` mask are reserved for processor-specific semantics. If meanings are specified, the processor supplement explains them.

If a permission bit is 0, that type of access is denied. Actual memory permissions depend on the memory management unit, which may vary from one system to another. Although all flag combinations are valid, the system may grant more access than requested. In no case, however, will a segment have write permission unless it is specified explicitly. The following table shows both the exact flag interpretation and the allowable flag interpretation. ABI-conforming systems may provide either.

Figure 5-4: Segment Permissions

Flags	Value	Exact	Allowable
<i>none</i>	0	All access denied	All access denied
PF_X	1	Execute only	Read, execute
PF_W	2	Write only	Read, write, execute
PF_W+PF_X	3	Write, execute	Read, write, execute
PF_R	4	Read only	Read, execute
PF_R+PF_X	5	Read, execute	Read, execute
PF_R+PF_W	6	Read, write	Read, write, execute
PF_R+PF_W+PF_X	7	Read, write, execute	Read, write, execute

For example, typical text segments have read and execute - but not write - permissions. Data segments normally have read, write, and execute permissions.

Segment Contents

An object file segment comprises one or more sections, though this fact is transparent to the program header. Whether the file segment holds one or many sections also is immaterial to program loading. Nonetheless, various data must be present for program execution, dynamic linking, and so on. The diagrams below illustrate segment contents in general terms. The order and membership of sections within a segment may vary; moreover, processor-specific constraints may alter the examples below. See the processor supplement for details.

Text segments contain read-only instructions and data, typically including the following sections described in Chapter 4. Other sections may also reside in loadable segments; these examples are not meant to give complete and exclusive segment contents.

Figure 5-5: Text Segment

.text
.rodata
.hash
.dynsym
.dynstr
.plt
.rel.got

Data segments contain writable data and instructions, typically including the following sections.

Figure 5-6: Data Segment

.data
.dynamic
.got
.bss

A PT_DYNAMIC program header element points at the .dynamic section, explained in [“Dynamic Section”](#) below. The .got and .plt sections also hold information related to position-independent code and dynamic linking. Although the .plt appears in a text segment in the previous table, it may reside in a text or a data segment, depending on the processor. See “Global Offset Table” and “Procedure Linkage Table” in this section of the processor supplement for details.

As [“Sections”](#) in Chapter 4 describes, the .bss section has the type SHT_NOBITS. Although it occupies no space in the file, it contributes to the segment's memory image. Normally, these uninitialized data reside at the end of the segment, thereby making p_memsz larger than p_filesz in the associated program header element.

Note Section

Sometimes a vendor or system builder needs to mark an object file with special information that other programs will check for conformance, compatibility, etc. Sections of type SHT_NOTE and program header elements of type PT_NOTE can be used for this purpose. The note information in sections and program header elements holds a variable amount of entries. In 64-bit objects (files with e_ident[EI_CLASS] equal to ELFCLASS64), each entry is an array of 8-byte words in the format of the target processor. In 32-bit objects (files with e_ident[EI_CLASS] equal to ELFCLASS32), each entry is an array of 4-byte words in the format of the target processor. Labels appear below to help explain note information organization, but they are not part of the specification.

Figure 5-7: Note Information

namesz
descsz
type
name
. . .
desc
. . .

namesz and name

The first namesz bytes in name contain a null-terminated character representation of the entry's owner or originator. There is no formal mechanism for avoiding name conflicts. By convention, vendors use their own name, such as XYZ Computer Company, as the identifier. If no name is present, namesz contains 0. Padding is present, if necessary, to ensure 8 or 4-byte alignment for the descriptor (depending on whether the file is a 64-bit or 32-bit object). Such padding is not included in namesz.

descsz and desc

The first descsz bytes in desc hold the note descriptor. The ABI places no constraints on a descriptor's contents. If no descriptor is present, descsz contains 0. Padding is present, if necessary, to ensure 8 or 4-byte alignment for the next note entry (depending on whether the file is a 64-bit or 32-bit object). Such padding is not included in descsz.

type

This word gives the interpretation of the descriptor. Each originator controls its own types; multiple interpretations of a single type value may exist. Thus, a program must recognize both the name and the type to recognize a descriptor. Types currently must be non-negative. The ABI does not define what descriptors mean.

To illustrate, the following note segment holds two entries.

Figure 5-8: Example Note Segment

	+0	+1	+2	+3	
namesz	7				No descriptor
descsz	0				
type	1				
name	x	y	z		
	c	o	\0	<i>pad</i>	
namesz	7				
descsz	8				
type	3				
name	x	y	z		
	c	o	\0	<i>pad</i>	
desc	<i>word 0</i>				
	<i>word 1</i>				

 The system reserves note information with no name (namesz==0) and with a zero-length name (name[0]=='\0') but currently defines no types. All other names must have at least one non-null character.

 Note information is optional. The presence of note information does not affect a program's ABI conformance, provided the information does not affect the program's execution behavior. Otherwise, the program does not conform to the ABI and has undefined behavior.

Thread-Local Storage

To permit association of separate copies of data allocated at compile-time with individual threads of execution, thread-local storage sections can be used to specify the size and initial contents of such data. Implementations need not support thread-local storage. A PT_TLS program entry has the following members:

Member	Value
p_offset	File offset of the TLS initialization image
p_vaddr	Virtual memory address of the TLS initialization image
p_paddr	reserved
p_filesz	Size of the TLS initialization image
p_memsz	Total size of the TLS template
p_flags	PF_R
p_align	Alignment of the TLS template

The *TLS template* is formed from the combination of all sections with the flag SHF_TLS. The portion of the TLS template that holds initialized data is the *TLS initialization image*. (The remaining portion of the TLS template is one or more sections of type SHT_NOBITS.)

[Previous](#) [Contents](#) [Next](#)

© 1997, 1998, 1999, 2000, 2001 The Santa Cruz Operation, Inc. All rights reserved.